

Player Role Consistency & Round Outcome Analysis in CS2

BTP Mid-Term Evaluation | Bachelor's Thesis Project

SACHIN KUMAR

UG-2 | Roll No: S20240010206

BTP Evaluation Presentation

Problem Statement & Objectives

Research Problem

Measuring Player Role Consistency and Its Impact on Match Outcome in CS2 — using behavioural signals extracted from raw .dem replay files, without manual annotation.

01 Feature Engineering

Extract 14 behavioural features per player per round from raw CS2 .dem files — kills, damage, utility, positioning, timing, and role-specific signals.

02 Unsupervised Role Detection

Cluster player-rounds into distinct tactical roles (Entry, Support, Lurker, IGL, AWPPer, Rifler) using an ensemble of KMeans, GMM, and HDBSCAN.

03 Impact on Outcomes

Quantify how role consistency correlates with round-win rates and match outcomes — players with consistent roles vs. "Flex" players.

Feature Engineering — 14 Behavioural Signals

Extracted per player per round from CS2 .dem files

⚔ Combat	💣 Utility & Support	📍 Positioning	🕒 Timing
kill_count Kills in round [0–5]	utility_used Grenades thrown [0–5]	distance_traveled Movement per round [0–5000]	first_engagement_rel Relative tick of 1st kill [0–1]
damage_dealt HP damage dealt [0–500]	flash_assists Flash-assist kills [0–5]	mean_isolation Avg dist from teammates [0–2000]	death_tick_rel Relative tick of death [0–1]
headshot_pct HS kill fraction [0–1]	trade_kills Trade kills [0–5]	zone_primary Map zone 1=CT / 2=mid / 3=T	survived Binary: alive at round end
multi_kill_round Binary: ≥3 kills in round			
opening_duel_won First kill / first death [-1,1]			
✅ G1 Validation Report: ALL 14 FEATURES VALID — ranges, caps, and scaled columns			

Current working pipeline :

The pipeline has two phases:

- Feature Engineering (G1): Parse demos → extract 14 behavioral metrics per player per round
- Clustering (G2): Group similar behaviors → label each group as a role

```
import subprocess, sys, os, glob, warnings, traceback, time
from pathlib import Path
from typing import Optional
from collections import Counter

def pip_install(pkg):
    subprocess.run([sys.executable, "-m", "pip", "install", pkg, "-q"], check=False)

for pkg in ["demoparser2", "tqdm", "hdbscan"]:
    pip_install(pkg)

import numpy as np
import pandas as pd
from scipy.spatial import cKDTree
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, confusion_matrix
from sklearn.manifold import TSNE
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from tqdm import tqdm

try:
    import hdbscan
    HDSCAN_AVAILABLE = True
except ImportError:
    HDSCAN_AVAILABLE = False
    warnings.warn("hdbscan not available - will use KMeans+GMM only.")

try:
    from demoparser2 import DemoParser
    PARSER_AVAILABLE = True
except ImportError:
    PARSER_AVAILABLE = False
    warnings.warn("demoparser2 not available.")

warnings.filterwarnings("ignore")

try:
    from google.colab import drive
    try:
        drive.flush_and_unmount()
    except:
        pass
    drive.mount('/content/drive', force_remount=True)
    time.sleep(3)
    print("📁 Google Drive mounted.")
except ImportError:
    print("i Not running on Colab - assuming Drive accessible.")

_BASE = "/content/drive/MyDrive/cs2-btp"
DEM_FOLDER = os.path.join(_BASE, "raw_dems")

if not os.path.isdir(DEM_FOLDER):
    raise FileNotFoundError(f"❌ DEM_FOLDER not found: {DEM_FOLDER}")

dem_check = glob.glob(os.path.join(DEM_FOLDER, "*.dem"))
if not dem_check:
    raise FileNotFoundError(f"❌ No .dem files in '{DEM_FOLDER}'")

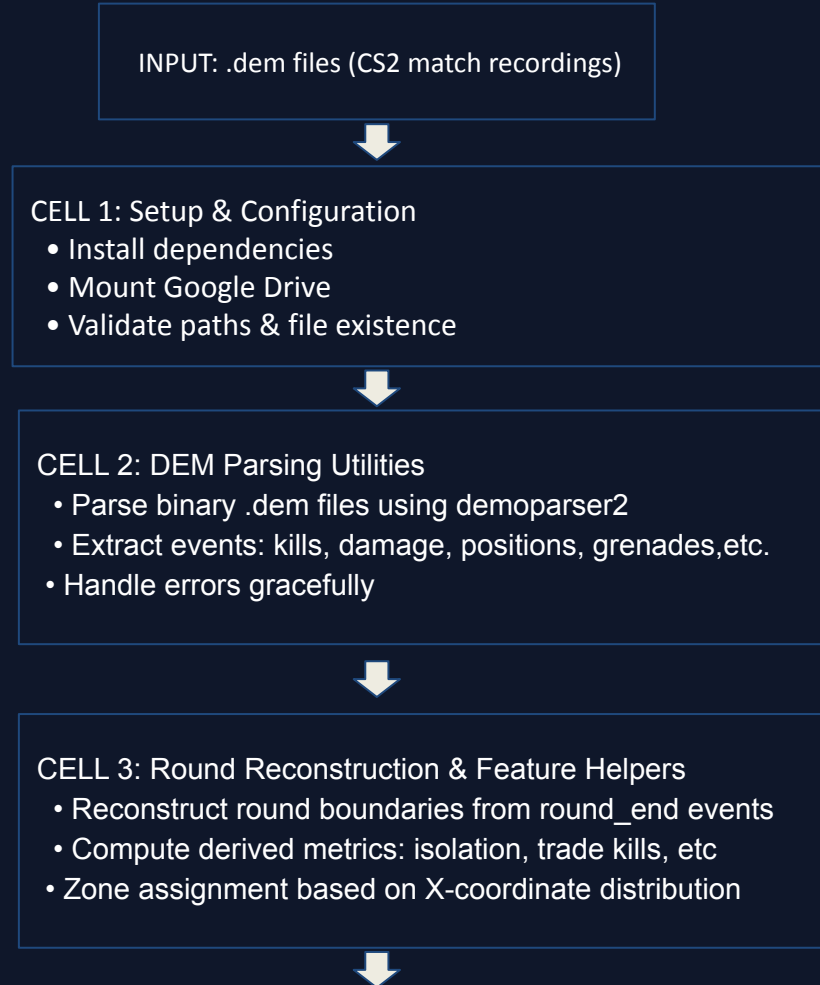
print(f"✅ DEM_FOLDER: {DEM_FOLDER} ({len(dem_check)} .dem files)")

OUTPUT_DIR = Path(os.path.join(_BASE, "processed"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

RANDOM_SEED = 42
MIN_TICK = 0
POS_DOWNSAMPLE_N = 64
CAP_DAMAGE = 500
CAP_KILLS = 5
CAP_UTILITY = 5
CAP_TRADE_KILLS = 5 # FIX 1: explicit cap constant (was missing - trade_kills reached 17)
CAP_DISTANCE = 5000
CAP_ISOLATION = 2000
TRADE_WINDOW_TICKS = 320

print(f"✅ Config loaded | HDSCAN: {'available' if HDSCAN_AVAILABLE else 'NOT available'}")
```

Pipeline :



Pipeline :



CELL 4: Feature Extraction

- For each (player, round, side) tuple:
- Aggregate kills, damage, utility usage
- Compute positional metrics
- Calculate timing-based features
- Output: DataFrame with 18 behavioral features



CELL 5: Preprocessing & Scaling

- Clip features to valid ranges
- Log-transform skewed distributions
- Validation report to catch data issues



G2: Clustering & Role Assignment

- Feature correlation analysis
- Optimal K selection (Silhouette + BIC)
- Ensemble clustering: KMeans + GMM + HDBSCAN
- Dimensionality reduction: PCA + t-SNE visualization
- Cluster profiling & radar charts
- Rule-based role assignment from cluster centroids
- Player-level aggregation & consistency scoring
- Optional validation against ground truth

CELL 1: SETUP & CONFIGURATION

Purpose

Initialize the environment, install dependencies, and configure file paths.

Key Code Pattern

```
1 def pip_install(pkg):  
2     subprocess.run([sys.executable, "-m", "pip", "install", pkg, "-q"], check=False)
```

- Why subprocess? Colab notebooks run in isolated environments; `pip` must be called as a subprocess, not imported.
- `check=False`: Prevents pipeline crash if package already installed.

```
try:  
    from google.colab import drive  
    drive.mount('/content/drive', force_remount=True)  
except ImportError:  
    print("i Not running on Colab – assuming Drive accessible.")
```

- Defensive programming: Works both in Colab and local environments.

```
1 TICK_DOWNSAMPLE = 64          # Sample position every 64 ticks (~1 second)  
2 POS_DOWNSAMPLE_N = 64         # Same for position parsing  
3 CAP_DAMAGE, CAP_KILLS = 500, 5 # Clip outliers for robust scaling  
4 TRADE_WINDOW_TICKS = 320      # ~5 seconds at 64 ticks/sec for trade detection
```

- Why cap features? Extreme values distort scaling and clustering. Capping creates robust features.
- Why downsample? Full tick data (~64Hz) is too dense; downsampling reduces compute while preserving behavior.

CELL 2: DEM PARSING UTILITIES

Understanding CS2 Demo Files

`.dem` files are binary recordings of CS2 matches containing:

- Player positions (X, Y, Z) every tick (~64 times/second)
- Events: kills, damage, grenade throws, round ends
- Metadata: map name, tick rate, player teams

Safe Parsing Wrapper

- Why wrap in try/except? Demo files can be corrupted or have missing events.
- Team name normalization: CS2 uses "TERRORIST"/"COUNTER_TERRORIST"; we map to "T"/"CT" for consistency.

```
def _safe_parse_event(parser, event, cols):  
    try:  
        df = parser.parse_event(event, player=cols)  
        if df is None or df.empty:  
            return pd.DataFrame()  
        # Normalize team names  
        for col in ["team_name", "attacker_team_name", "user_team_name"]:  
            if col in df.columns:  
                df[col] = df[col].map(lambda x: TEAM_MAP.get(str(x).upper(), str))  
        return df  
    except Exception:  
        return pd.DataFrame()
```

Position Sampling Strategy

- Why not all ticks? 10 minutes of gameplay at 64 ticks/sec = 38,400 ticks per player. Downsampling by 64x reduces to ~600 samples/round—enough for behavioral analysis.

```
1 sample_ticks = list(range(MIN_TICK + 1, max_tick, POS_DOWNSAMPLE_N))  
2 positions = _safe_parse_ticks(parser, ["X", "Y", "Z", "team_name"], sample_ticks)
```


CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

Round Reconstruction: Turning Ticks into Rounds

A CS2 match has 24–30+ rounds, but the demo file doesn't explicitly label events with round numbers. Instead, it records `round_end` events with their tick. We must reverse-engineer round boundaries from these ticks.

`reconstruct_rounds()`

```
def reconstruct_rounds(round_ends_df, max_fallback_tick=150_000):
    if round_ends_df.empty or "tick" not in round_ends_df.columns:
        return pd.DataFrame({"round_end_tick": [max_fallback_tick], "round_num": [1]})
    ticks = sorted(round_ends_df["tick"].dropna().unique().tolist())
    return pd.DataFrame({"round_end_tick": ticks, "round_num": range(1, len(ticks) + 1)})
```

What it does:

1. Takes the raw `round_end` events (which just have tick numbers)
2. Sorts them chronologically
3. Assigns round numbers 1, 2, 3, ...

Example output:

	round_end_tick	round_num
0	12800	1
1	21500	2
2	33200	3
...		
28	210000	29

Fallback: If the demo has no `round_end` events (corrupt file), it creates a single "round" spanning the entire match.

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

`assign_round_num()` – Mapping Ticks to Rounds

```
def assign_round_num(tick_series, round_map):  
    bins = [0] + round_map["round_end_tick"].tolist()  
    labels = round_map["round_num"].tolist()  
    assigned = pd.cut(tick_series, bins=bins, labels=labels, right=True)  
    return pd.to_numeric(assigned, errors="coerce").fillna(1).astype(int)
```

The problem: Every kill, damage, and position event has a `tick` value. We need to know which round that tick belongs to.

The solution: `pd.cut()` creates bins from the round end ticks and assigns each event tick to its bin (round).

Visual explanation:

Round boundaries:	0	—	12800	—	21500	—	33200	—	...
		R1		R2		R3		R4	
Kill at tick 15000:				← here					→ Round 2
Kill at tick 8000:		← here							→ Round 1
Position at 25000:						← here			→ Round 3

`pd.cut()` does this automatically:

- `bins = [0, 12800, 21500, 33200, ...]` – the boundaries
- `labels = [1, 2, 3, ...]` – what to call each bin
- `right=True` – a tick exactly AT 12800 belongs to round 1 (inclusive right boundary)

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

`compute_zone_primary()`

```
def compute_zone_primary(mean_x, all_x):  
    """  
    Classify a player's mean X position into map zones using the global  
    X distribution (all ticks, all players, whole match set) as the refer  
    Returns 1 (CT/right side), 2 (mid), or 3 (T/left side).  
    """  
    if pd.isna(mean_x) or len(all_x) < 10:  
        return 2  
    p33 = np.nanpercentile(all_x, 33)  
    p66 = np.nanpercentile(all_x, 66)  
    if mean_x < p33:  
        return 3  
    elif mean_x > p66:  
        return 1  
    else:  
        return 2
```

Why Percentiles Instead of Fixed Thresholds?

Each CS2 map has completely different coordinate systems:

- **Dust2:** X ranges from roughly -2500 to +2500
- **Mirage:** X ranges from roughly -3000 to +1500
- **Nuke:** X ranges from roughly -1500 to +2000

If we used fixed thresholds like "X < -1000 = T side", it would be correct on Dust2 but wrong on Mirage.

The percentile approach is map-adaptive:

1. Collect ALL X positions from ALL players in the entire match (`all_x`)
2. Calculate the 33rd and 66th percentiles — these naturally divide the map into thirds
3. Players in the bottom third → T-side (zone 3), middle third → mid (zone 2), top third → CT-side (zone 1)

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

Isolation measures how far a player positions themselves from their teammates. High isolation = playing alone (lurking). Low isolation = staying with the pack (supporting).

`mean_isolation()`

```
def mean_isolation(positions_df, player_name, round_num):
    rnd_pos = positions_df[positions_df["round_num"] == round_num]

    # Get this player's positions
    own = rnd_pos[rnd_pos[name_col] == player_name]
    own_xy = own[["X", "Y"]].dropna().values

    # Get this player's team
    own_team = own["team_name"].iloc[0]

    # Get teammate positions (same team, different player)
    mates = rnd_pos[(rnd_pos["team_name"] == own_team) &
                    (rnd_pos[name_col] != player_name)]
    mate_xy = mates[["X", "Y"]].dropna().values

    # Build a KD-tree from teammate positions
    tree = cKDTree(mate_xy)

    # For each of the player's positions, find the nearest teammate
    dists, _ = tree.query(own_xy, k=1)

    # Return the average nearest-teammate distance
    return float(np.mean(dists))
```

Step-by-Step Example

Imagine Round 5 on Dust2. Player "s1mple" is on the T-side with 4 teammates.

s1mple's positions (sampled every second):

Tick 1: (1200, 800) – near T-spawn
Tick 2: (1400, 600) – moving toward Long
Tick 3: (1800, 300) – Long Doors
Tick 4: (2200, 100) – Long A site approach

Teammate positions (all 4 teammates, all ticks combined):

(600, 400), (650, 380), (700, 450), ... – teammates at B/mid

The KD-tree is built from ALL teammate positions (maybe 200+ data points). For each of s1mple's 4 positions, it finds the nearest teammate:

Tick 1: nearest teammate at 580 units away
Tick 2: nearest teammate at 720 units away
Tick 3: nearest teammate at 1100 units away ← getting more isolated
Tick 4: nearest teammate at 1500 units away ← very isolated

Mean isolation = $(580 + 720 + 1100 + 1500) / 4 = 975$ units

A support player staying with the team might have mean isolation of ~200 units. A lurker might have ~1200+ units.

Why Only Same-Team Teammates?

We only measure distance from **teammates**, not enemies. A player could be far from teammates but close to enemies (flanking). We want to capture "how far from your own team are you playing?" – that's what distinguishes lurkers from support players.

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

```
def distance_traveled(pos_rows):  
    xy = pos_rows[["X", "Y"]].dropna().values  
    if len(xy) < 2:  
        return 0.0  
    return float(np.sum(np.linalg.norm(np.diff(xy, axis=0), axis=1)))
```

What This Computes

Total path length — the sum of Euclidean distances between consecutive position samples.

Mathematical Breakdown

Given positions: $P_1=(100,200)$, $P_2=(150,250)$, $P_3=(300,250)$, $P_4=(350,300)$

```
np.diff(xy, axis=0)    # Differences between consecutive positions  
# = [(50,50), (150,0), (50,50)]  
  
np.linalg.norm(..., axis=1)    # Euclidean distance of each step  
# = [ $\sqrt{50^2+50^2}$ ,  $\sqrt{150^2+0^2}$ ,  $\sqrt{50^2+50^2}$ ]  
# = [70.7, 150.0, 70.7]  
  
np.sum(...)    # Total distance  
# = 291.4 game units
```

Why 2D Not 3D?

We ignore the Z axis (height) because:

1. Most CS2 maps are played on a single elevation level
2. Vertical movement (jumping, climbing) is noise — it doesn't indicate tactical positioning
3. Ladders/stairs would inflate distance for players who simply change floors

What Distance Tells Us About Roles

Role	Typical Distance	Why
Lurker	2000–4000+	Covers large map areas, flanking through different routes
Entry	1500–3000	Rushes toward sites aggressively
Support	800–1500	Follows the entry, stays closer to starting position
AWPer	500–1200	Holds angles from static positions, moves less
IGL	800–1500	Moderate movement, directing rather than fragging

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

What Is a Trade Kill?

In CS2, a "trade kill" happens when:

1. Your teammate dies
2. Within **5 seconds**, you kill the enemy who killed them (or any enemy nearby)

Trade kills are a key metric for **support players** — they're the ones who "trade" for fallen teammates, ensuring the team doesn't lose engagements for free.

compute_trade_kills()

```
def compute_trade_kills(kills_df, player_name, round_num, side):
    rnd_kills = kills_df[kills_df["round_num"] == round_num]

    # Find teammate deaths (same team, not this player)
    mate_deaths = rnd_kills[
        (rnd_kills["user_team_name"] == side) &
        (rnd_kills["user_name"] != player_name)
    ]

    # Find this player's kills
    player_kills = rnd_kills[rnd_kills["attacker_name"] == player_name]

    # Count trades: player kill within 320 ticks (5 sec) of a mate death
    trades = 0
    for _, pk in player_kills.iterrows():
        for _, md in mate_deaths.iterrows():
            if 0 < (pk["tick"] - md["tick"]) <= TRADE_WINDOW_TICKS:
                trades += 1
                break    # Each kill can only be one trade

    return min(trades, CAP_TRADE_KILLS)    # FIX 1: cap at 5
```

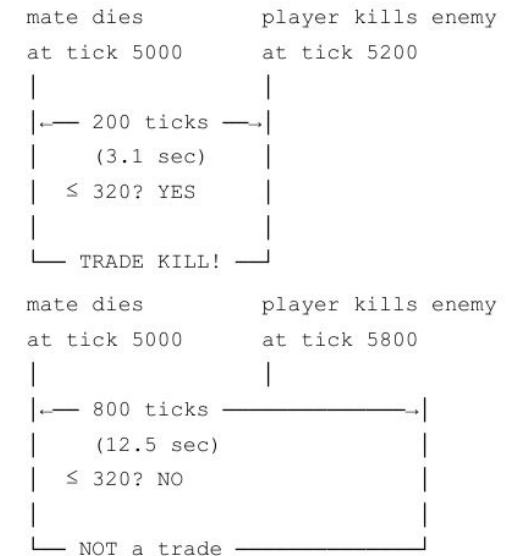
The **break** Statement Is Critical

```
for _, pk in player_kills.iterrows():    # For each of the player's kill
    for _, md in mate_deaths.iterrows():    # Check all teammate deaths
        if 0 < (pk["tick"] - md["tick"]) <= 320:
            trades += 1
            break    # ← This kill is a trade. Stop checking more deaths.
```

Without **break**, a single kill that happens after 2 teammate deaths (both within 5 seconds) would be counted as 2 trades. With **break**, it's correctly counted as 1 trade.

Time Window Visualization

Timeline (ticks):



CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

What Is an Opening Duel?

The **first kill** of each round is the "opening duel." The player who gets the opening kill has won the opening duel (+1). The player who dies first has lost it (-1). All other 8 players get 0.

```
def compute_opening_duel(kills_df, player_name, round_num):
    rnd = kills_df[kills_df["round_num"] == round_num].sort_values("tick")
    if rnd.empty:
        return 0
    first = rnd.iloc[0]  # The first kill of the round
    if first.get("attacker_name") == player_name:
        return 1  # Won the opening duel
    if first.get("user_name") == player_name:
        return -1  # Lost the opening duel (died first)
    return 0  # Wasn't involved
```

Why Opening Duels Matter for Roles

Role	Expected Opening Duel Pattern
Entry	Frequently +1 or -1 (always taking the first fight)
AWPer	Often +1 (gets first pick from holding an angle)
Lurker	Usually 0 (not involved in early fights)
Support	Usually 0 (following behind the entry)
IGL	Usually 0 (directing, not fragging first)

Entry fraggers have the most volatile opening duel stats – they either win or lose the first fight.

Support/Lurker/IGL players rarely participate in the opening duel.

CELL 3: ROUND RECONSTRUCTION & FEATURE HELPERS

Why 14 Features? Why These Specific Ones?

The 14 features were chosen to capture **distinct behavioral dimensions** that differentiate CS2 roles:

Dimension	Features	What It Measures
Aggression	damage_dealt, kill_count, multi_kill_round	Raw fragging power
Timing	first_engagement_rel, death_tick_rel	When in the round the player acts
Positioning	mean_isolation, distance_traveled, zone_primary	Where the player plays
Teamplay	utility_used, trade_kills, flash_assists	How much they support teammates
Precision	headshot_pct	Weapon accuracy profile (rifler vs AWPPer)
Survivability	survived	Whether the player lives through the round
Initiative	opening_duel_won	Who takes/wins the first fight

Each feature captures a different aspect of gameplay. Together, they create a 14-dimensional behavioral fingerprint for each player-round.

CELL 4: Feature Extraction

`extract_features_from_dem()`

This is the most complex function in the pipeline. It takes a single `.dem` file and produces a DataFrame where **each row = one player in one round** with 14 behavioral features.

High-Level Flow

```
.dem file
|
▼
parse_dem_file() → kills, damage, positions, grenades, round_ends
|
▼
reconstruct_rounds() → round_map (round boundaries)
|
▼
assign_round_num() applied to all DataFrames → each event tagged with
round_num
|
▼
Collect unique (player, round, side) combos
|
▼
For EACH combo: compute 14 features
|
▼
Output: DataFrame with ~300 rows (10 players × 30 rounds)
```

What's a "combo"? A tuple like `("Zyw0o", 5, "CT")` meaning "Zyw0o in round 5, playing on CT side."

CELL 5 : Preprocess and Scaling :

Three Feature Categories

```
LOG_FEATURES      = ["damage_dealt", "distance_traveled", "mean_isolation"]
COUNT_FEATURES   = ["kill_count", "utility_used", "trade_kills", "flash_as
BINARY_FEATURES   = ["multi_kill_round", "survived"]
PASSTHROUGH      = ["first_engagement_rel", "death_tick_rel", "zone_primar
                    "headshot_pct", "opening_duel_won"]
```

Why Preprocess?

Raw features have wildly different scales:

- `damage_dealt` : 0 to 500 (large numbers)
- `headshot_pct` : 0 to 1.0 (tiny numbers)
- `distance_traveled` : 0 to 5000 (huge numbers)
- `survived` : 0 or 1 (binary)

If we feed these directly to KMeans, `distance_traveled` would dominate because its values are ~5000× larger than `headshot_pct`. KMeans uses Euclidean distance, and a difference of 100 in `distance_traveled` would "matter" 500× more than a difference of 0.2 in `headshot_pct`. This is wrong — both features should have equal influence.

Category	Transform	Why
LOG features	<code>log1p()</code> → <code>StandardScaler</code>	These features are right-skewed (many low values, few high values). Log transform makes them more normal
COUNT features	<code>StandardScaler</code> only	Small integer ranges (0–5), already roughly symmetric
BINARY features	<code>StandardScaler</code> only	0/1 values, scaling centers them at 0
PASSTHROUGH	No transform (already [0,1] or small range)	Already on a reasonable scale

CELL 5 : Preprocess and Scaling :

Log Transform: `np.log1p()`

What `log1p` Does

`log1p(x) = ln(x + 1)` — the natural logarithm of (x + 1).

The "+1" is critical: `ln(0)` is undefined ($-\infty$), but `ln(0 + 1) = ln(1) = 0`. So `log1p(0) = 0`, which is safe.

Why Log Transform?

`damage_dealt` has a heavily right-skewed distribution:

Without log:

0-50		(70% of values)
50-100		(20%)
100-200		(7%)
200-500		(3%) ← outliers dominate

With `log1p`:

0-2		(35%)
2-3.5		(35%)
3.5-5		(22%)
5-6.2		(8%) ← much more balanced

Log compresses large values and spreads out small values, making the distribution more **symmetric/normal**. This is important because `StandardScaler` assumes roughly normal distributions.

Concrete Example

damage values: [0, 15, 30, 45, 80, 120, 200, 450]

After `log1p`: [0, 2.77, 3.43, 3.83, 4.39, 4.80, 5.30, 6.11]
← Much more evenly spaced

```
scaler = StandardScaler()
scaled_vals = scaler.fit_transform(df[to_scale].fillna(0))
```

What `StandardScaler` Does

For each feature column, it transforms every value to a **z-score**:

$$z = (x - \mu) / \sigma$$

Where:

- x = the original value
- μ = the mean of that feature across all rows
- σ = the standard deviation of that feature

After Scaling

- Mean of each feature = 0
- Standard deviation = 1
- A value of +2.0 means "2 standard deviations above average"
- A value of -1.5 means "1.5 standard deviations below average"

Why This Matters for Clustering

Before scaling:

Player A: damage=300, distance=4000
Player B: damage=280, distance=100

Euclidean distance = $\sqrt{((300-280)^2 + (4000-100)^2)} = \sqrt{(400 + 15,210,000)} \approx 3900$
↑ distance dominates!

After scaling (assuming $\mu_{\text{dmg}}=150$, $\sigma_{\text{dmg}}=80$, $\mu_{\text{dist}}=2000$, $\sigma_{\text{dist}}=1500$):

Player A: damage_scaled=(300-150)/80=1.88, distance_scaled=(4000-2000)/1500=1.33
Player B: damage_scaled=(280-150)/80=1.63, distance_scaled=(100-2000)/1500=-1.27

Euclidean distance = $\sqrt{((1.88-1.63)^2 + (1.33-(-1.27))^2)} = \sqrt{(0.0625 + 6.76)} \approx 2.61$

features contribute

↑ both

CELL 5 : Preprocess and Scaling :

Validation Report

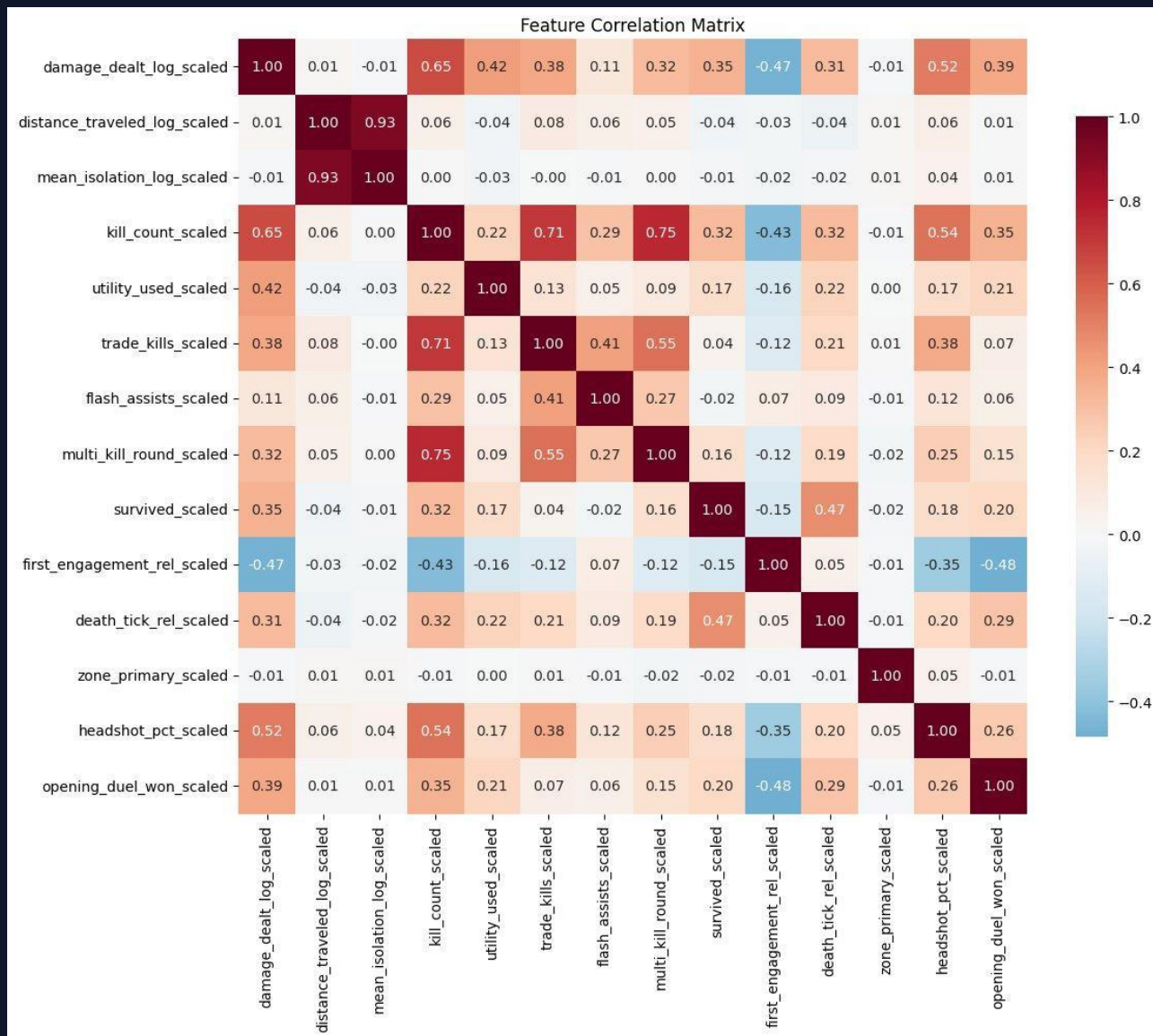
```
raw_check = {  
    "damage_dealt":      (0, CAP_DAMAGE),  
    "kill_count":        (0, CAP_KILLS),  
    ...  
}  
for col, (lo, hi) in raw_check.items():  
    mn, mx = clean_df[col].min(), clean_df[col].max()  
    ok = "✓" if (mn >= lo - 1e-6 and mx <= hi + 1e-6) else "✗"
```

This sanity check verifies that every feature is within its expected range after preprocessing. If any feature fails:

- Capping might have been missed
- A bug in feature computation produced out-of-range values
- Demo parsing returned unexpected data

The `1e-6` tolerance accounts for floating-point precision errors (e.g., 0.99999999 vs 1.0).

Feature Correlation Analysis



Notable Correlations

$r = 0.93$

distance_traveled & mean_isolation — high collinearity; both kept (log-scaled)

$r = 0.75$

kill_count & multi_kill_round — expected; kill count drives multi-kill flag

$r = 0.71$

kill_count & trade_kills — players who get kills also get more trades

$r = -0.47$

damage_dealt & first_engagement_rel — early engagers deal more damage

$r = -0.48$

opening_duel_won & first_engagement_rel — winning entry = engaging early

$r = 0.47$

survived & death_tick_rel — survivors have higher late death tick (didn't die)

→ zone_primary is near-zero correlated with everything — clean orthogonal feature

CELL 5 : Preprocess and Scaling :

Validation Report

```
raw_check = {  
    "damage_dealt":      (0, CAP_DAMAGE),  
    "kill_count":        (0, CAP_KILLS),  
    ...  
}  
for col, (lo, hi) in raw_check.items():  
    mn, mx = clean_df[col].min(), clean_df[col].max()  
    ok = "✓" if (mn >= lo - 1e-6 and mx <= hi + 1e-6) else "✗"
```

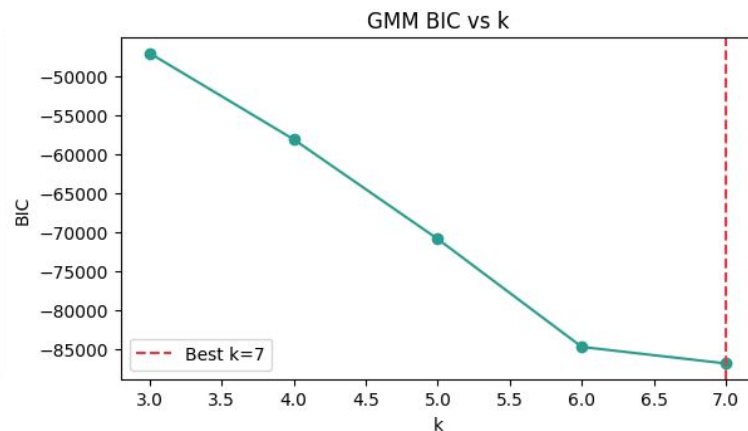
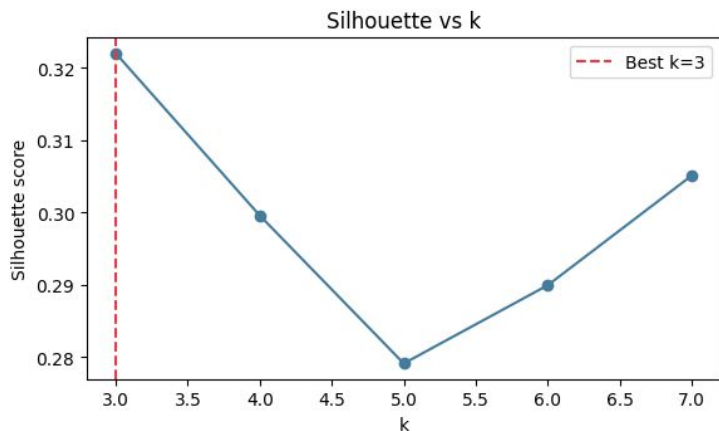
This sanity check verifies that every feature is within its expected range after preprocessing. If any feature fails:

- Capping might have been missed
- A bug in feature computation produced out-of-range values
- Demo parsing returned unexpected data

The `1e-6` tolerance accounts for floating-point precision errors (e.g., 0.99999999 vs 1.0).

K Selection & Cluster Visualisation

K Selection | Chosen K = 5



K = 5 Selected

Silhouette

Best k=3 (score ≈ 0.324)

GMM BIC

Best k=7 (BIC still falling)

Chosen K

$|3-7| > 1 \rightarrow \text{average} \rightarrow \text{round} \rightarrow 5$

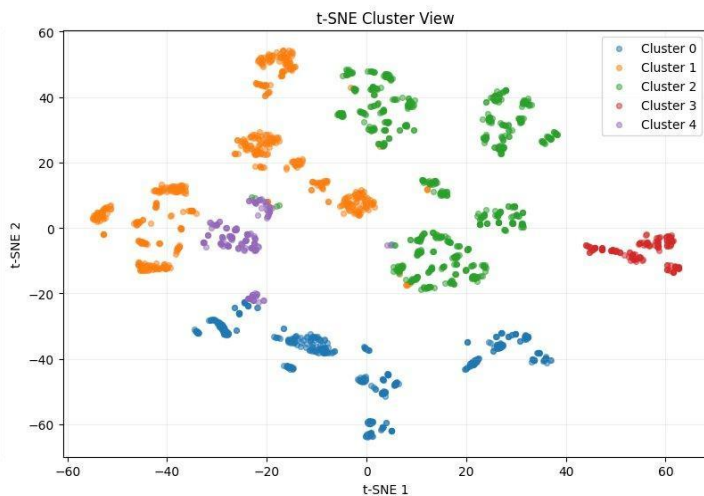
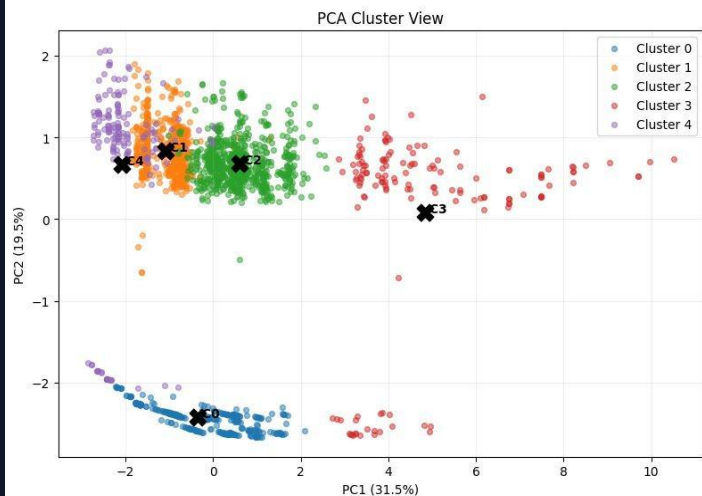
PCA var.

PC1=31.5% + PC2=19.5% = 51%

t-SNE

5,000 pts, perplexity=30, 1000 iter

CS2 Player Roles — Dimensionality Reduction



Silhouette favors parsimony. It says: *"3 broad archetypes capture the clearest behavioral splits."*

BIC favors expressiveness. It says: *"Player behavior has nuanced subtypes; 7 clusters explain the data better than 3, even after penalizing complexity."*

The heuristic: Since $|3 - 7| = 4 > 1$, the pipeline takes the midpoint: $(3 + 7) / 2 = 5$. This is a pragmatic compromise, not a statistically optimal rule. It acknowledges:

K=3 might oversimplify (miss tactical niches)

K=7 might overfit (split natural variation into artificial sub-clusters)

K=5 balances interpretability with granularity.

Ensemble Clustering Methodology

Three algorithms vote on each point — majority wins

K-Means

K=5 | n_init=10

- Fast, globally convergent
- Reference for alignment
- Provides cluster centroids for PCA viz

GMM

Full covariance | n_init=5

- Soft probabilistic assignments
- BIC for optimal K selection
- gmm_max_prob confidence per point

HDBSCAN

*min_cluster_size=20
min_samples=5*

- Density-based, finds irregular shapes
- Noise points (-1) reassigned via KMeans
- Lowered threshold for granularity (FIX 6)



Majority Vote Consensus

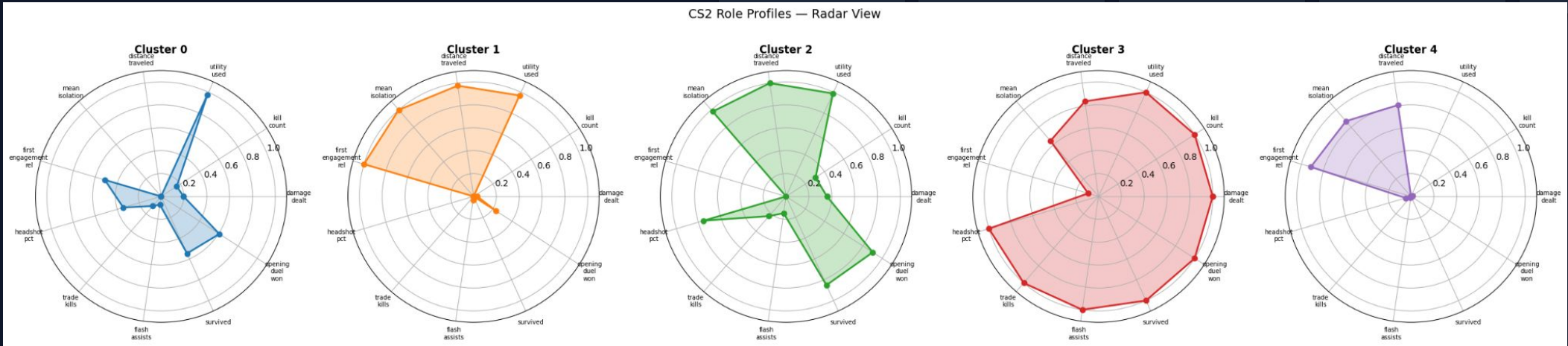
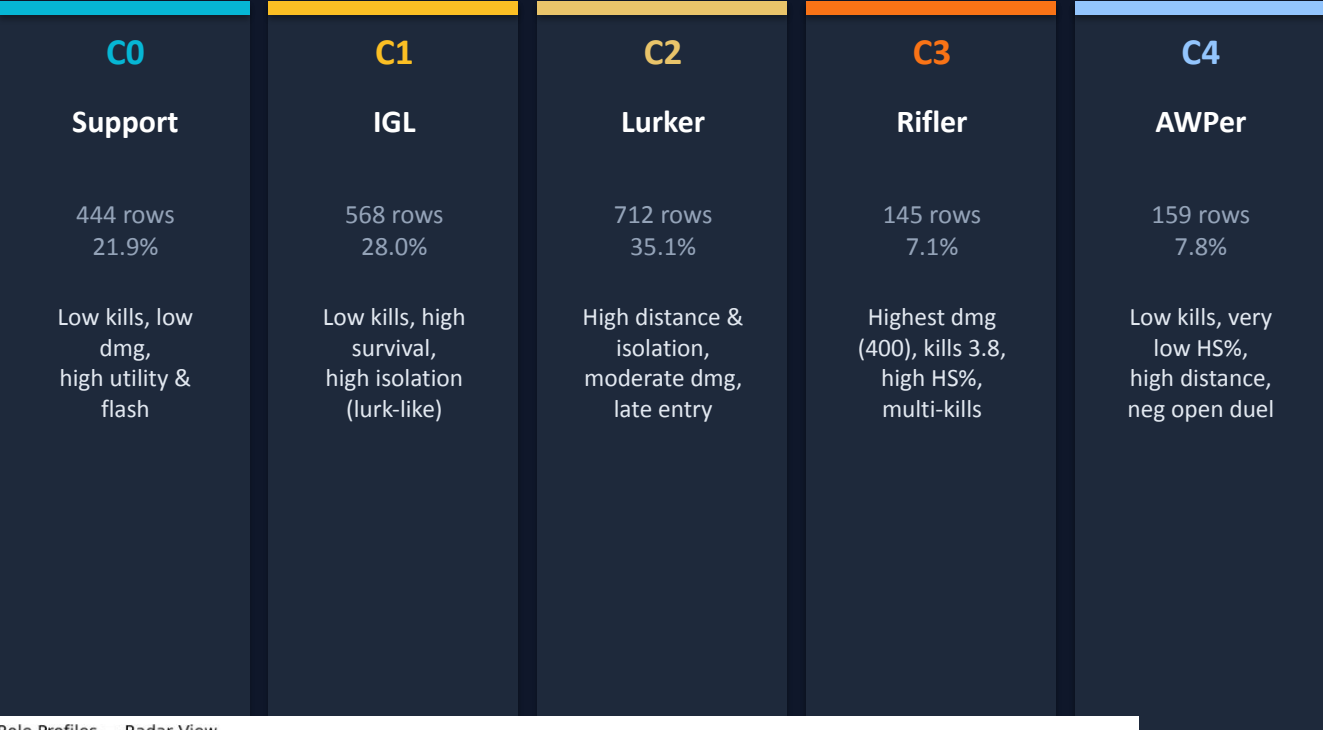
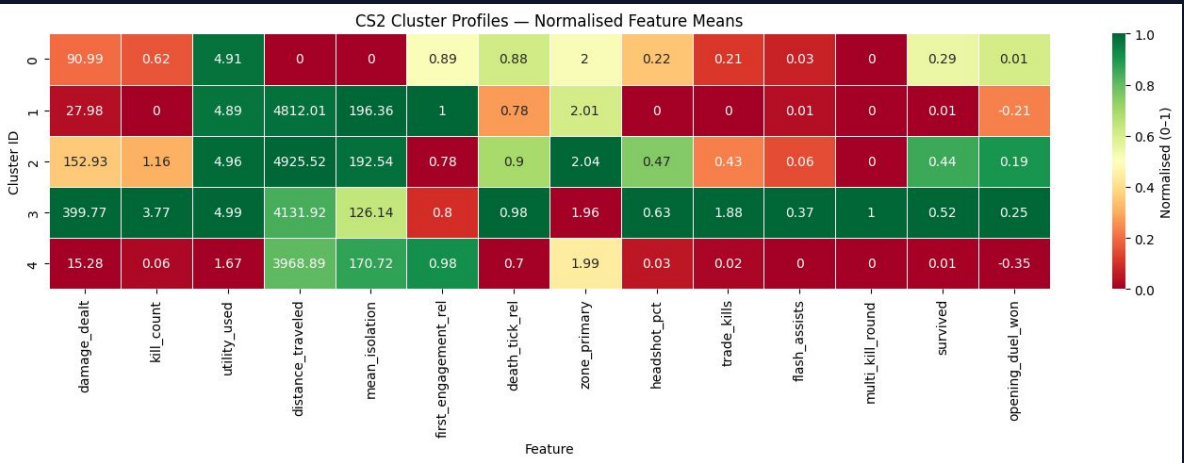


Ensemble Agreement: 38.7% | Optimal K = 5 (Silhouette → 3, BIC → 7, chosen: mean rounded = 5)

For each point: [KMeans label, GMM-aligned label, HDBSCAN-aligned label] → Counter.most_common(1). GMM & HDBSCAN are Hungarian-aligned to KMeans before voting.

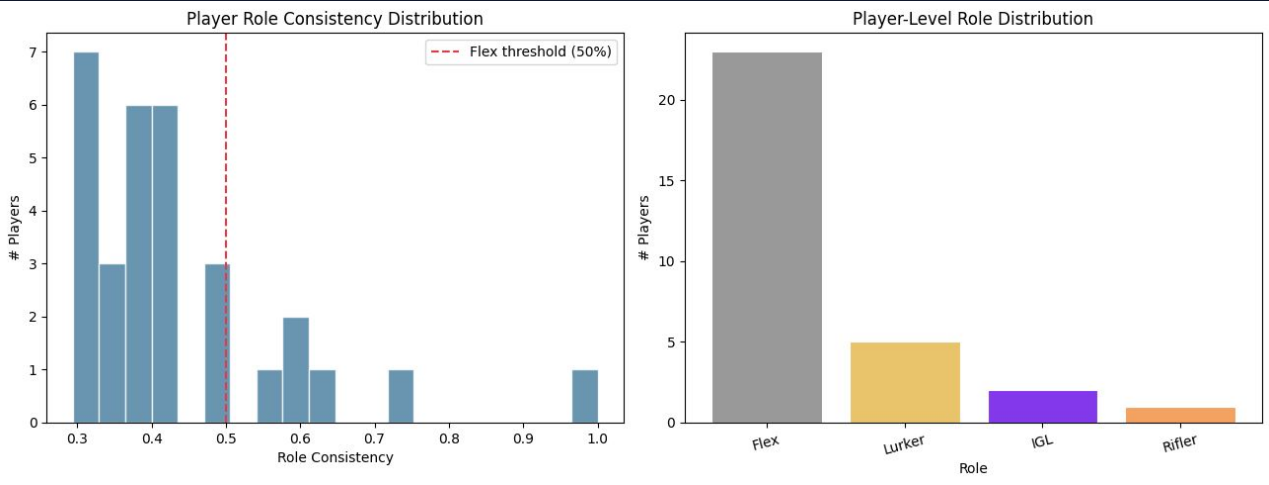
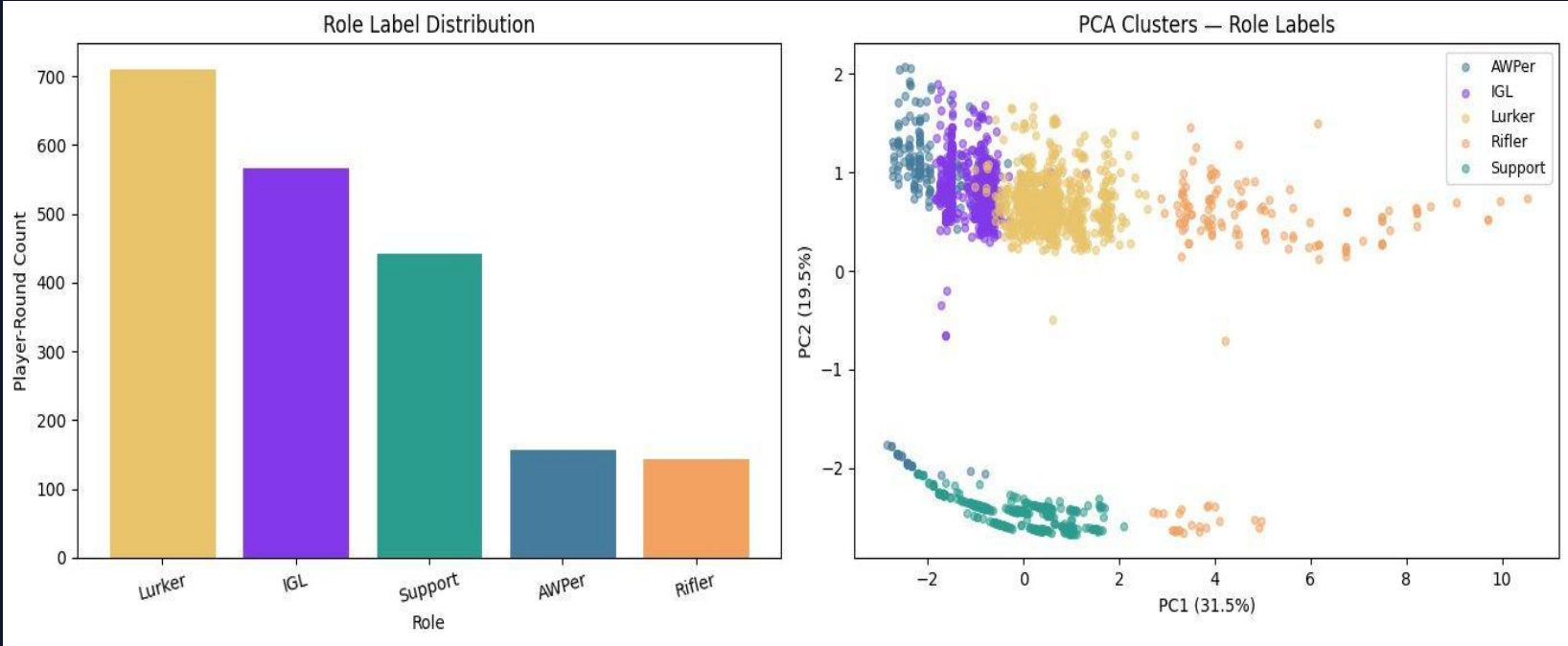
HDBSCAN noise points (-1) inherit the KMeans label for that point before alignment.

Cluster Profiles & Role Assignment



Player-Level Role Analysis

31 unique players | Flex threshold = 50% consistency



Notable Players

Lurker

hyped	73% (30 rds)
JDC	63% (30 rds)
LNZ	59% (32 rds)

IGL

SunPayus	50% (32 rds)
xfI0ud	50% (30 rds)

Rifler

zweih	100% (4 rds)
-------	--------------

Flex (< 50%)

tN1R	47% (36 rds)
chopper	43% (106 rds)
ropz	42% (123 rds)

23 of 31 players fall below 50% consistency → classified Flex

Player-Level Role Analysis

Issues :

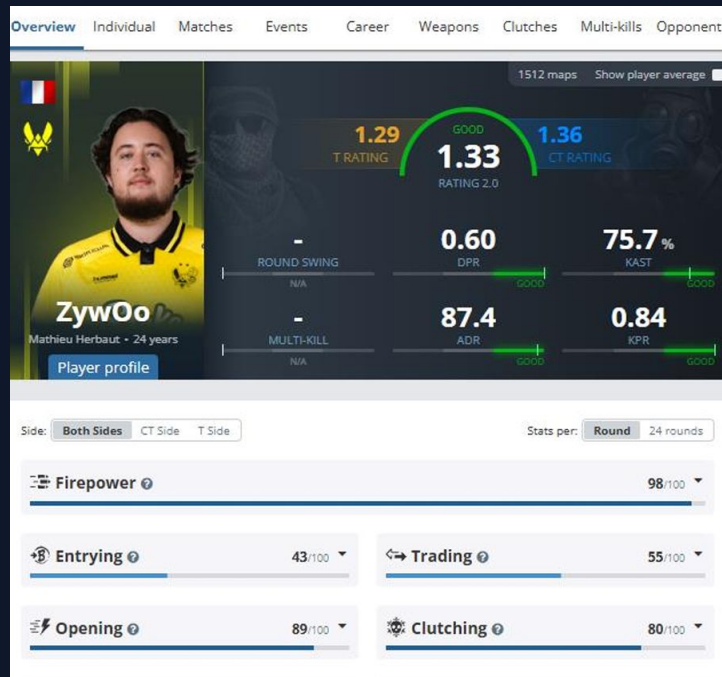
1. Data corruption in cluster profiles : Many features show exactly 0 which suggests missing data or wrong column selection
2. Role Labels Don't Match Behavioral Profiles
3. Cluster Imbalance
4. Low Player Consistency (74% labeled "Flex")

Possible fixes :

1. Increase the number of .dem files (number of matches in the database) - which might remove the map based role selection biases
2. Introduce better feature selection like HLTV's Rating 2.0
3. Look at options other than hard capped community based roles, potentially behavior based analogy which already implemented in the research paper- FEATURE ENGINEERING FOR ROLE ASSESSMENT IN COUNTER-STRIKE 2 Yurii Kuzhii* , Yuriy Furgala) with successful results.

Player-Level Role Analysis - Literature Review

FEATURE ENGINEERING FOR ROLE ASSESSMENT IN COUNTER-STRIKE 2 Yurii Kuzhii* , Yuriy Furgala, implements behavior based categorization of players by using the features available in HLTV's rating 2.0



The results of this study demonstrate that player roles in CS2 can be effectively identified using behavioral features extracted from publicly available statistical sources, particularly HLTV.org.

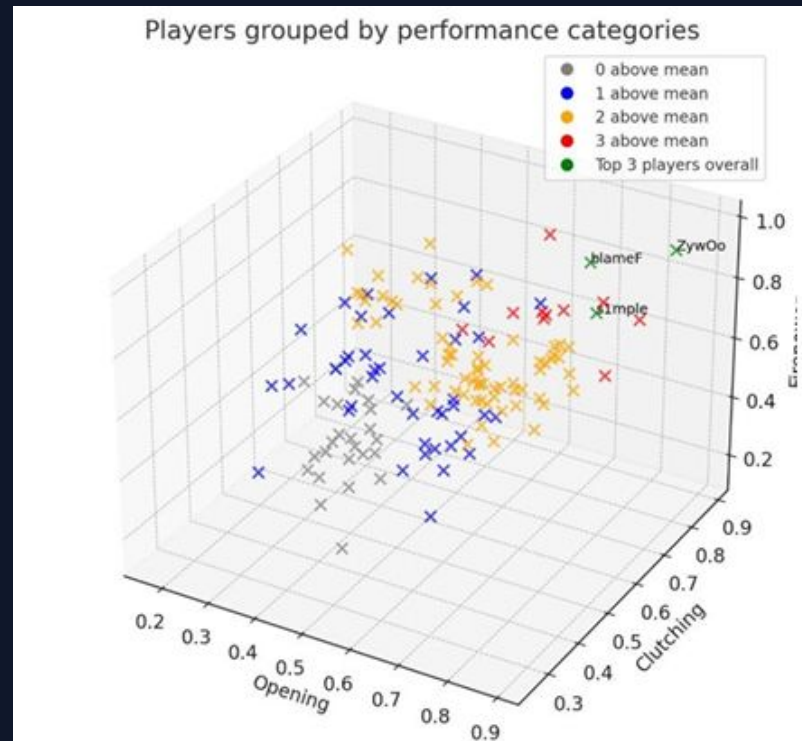


Fig. 2. Visualization of the firepower, clutching, and opening attributes.

As shown in Fig. 2, no players exhibit simultaneously high values across all three metrics. This can be explained by the fact that trading and clutching represent opposing behavioral attributes: a player who operates alone cannot effectively participate in trade situations.

Goals for the upcoming semester

The majority of the work in the current study revolves around categorising players into role/behavior based categories. Thus, once we implement a successful pipeline to do so, we can instantly run our analysis on the data.

Henceforth, moving forward our main focus will be to create a robust pipeline that stress tested in player classification and then perform our respective consistency or role based studies on it.

THANK YOU

SACHIN KUMAR

S20240010206 (UG 2)